

JSON

JAVASCRIPT OBJECT NOTATION

Formato JSON

¿QUÉ ES? ¿PARA QUÉ SIRVE?

En el universo de la programación, el intercambio de datos es un aspecto fundamental y uno de los formatos más utilizados para tal fin es: JSON.

JSON, cuyo nombre corresponde a las siglas JavaScript Object Notation o Notación de Objetos de JavaScript, es un formato ligero de intercambio de datos, que resulta sencillo de leer y escribir para los programadores y simple de interpretar y generar para las máquinas.

JSON es un formato de texto **completamente independiente del lenguaje de programación que se esté utilizando.**

Formato JSON

¿QUÉ ES? ¿CÓMO SURGIÓ? ¿PARA QUÉ SIRVE?

A principios de la década de los 90 surgió el problema de que máquinas diferentes pudieran entenderse entre sí.

Las máquinas utilizaban diferentes sistemas operativos y sus programas estaban escritos en diferentes lenguajes de programación. Una de las soluciones fue crear el estándar XML.

Sin embargo, XML presentaba problemas sobre todo cuando se trataba de trabajar con gran volumen de datos, puesto que el procesamiento se volvía lento.

Surgieron entonces intentos para definir formatos que fueran más ligeros y rápidos para el intercambio de información.

Formato JSON

¿QUÉ ES? ¿CÓMO SURGIÓ? ¿ PARA QUÉ SIRVE?

Uno de ellos fue JSON, promovido y popularizado a principios de los 2000 por Douglas Crockford, un programador conocido como el 'gurú' de JavaScript.

Desde entonces JSON se caracteriza por reducir el tamaño de los archivos y el volumen de datos que es necesario transmitir.

Por ello fue adquiriendo popularidad hasta convertirse en un estándar y uno de los formatos de intercambios de datos más utilizados en aplicaciones web.

Esto no significa que XML haya dejado de utilizarse, en la actualidad ambos se emplean para el intercambio de datos.

Formato JSON

¿QUÉ ES? ¿CÓMO SURGIÓ? ¿ PARA QUÉ SIRVE?

Pese a su nombre, no es parte de JavaScript, de hecho, es un estándar basado en texto plano para el intercambio de datos, por lo que se usa en muchos sistemas que requieren mostrar o enviar información para ser interpretada por otros sistemas.

LENGUAJES CON SOPORTE para JSON: con el tiempo prácticamente todos los lenguajes de programación han introducido soporte para el intercambio de datos basado en JSON.

- C
 - C++
 - C#
 - Java
 - JavaScript
 - Perl
 - PHP
 - Python
- Entre otros

Formato JSON

¿QUÉ ES? ¿CÓMO SURGIÓ? ¿ PARA QUÉ SIRVE?

Una de las características de JSON, al ser un formato que es independiente de cualquier lenguaje de programación, es que los servicios que comparten información por este método no necesitan hablar el mismo idioma, es decir,

- el emisor puede ser Java y el receptor Python, pues cada uno tiene su propia librería para codificar y decodificar cadenas en este formato.
- En nuestro caso el emisor va a ser PHP y el receptor JavaScript

Dichas propiedades hacen de JSON un formato de intercambio de datos ideal para usar con **API REST** o **AJAX**.

Formato JSON

¿QUÉ ES? ¿CÓMO SURGIÓ? ¿ PARA QUÉ SIRVE?

Prácticamente todos los lenguajes de programación han introducido soporte para el intercambio de datos basado en JSON.

Soporte de PHP para trabajar con JSON:

Funciones nativas de JSON de php

`json_decode()` — toma una cadena codificada en JSON como su primer parámetro y la transforma en una variable de PHP.

`json_encode()` — convertirá una matriz PHP en una cadena codificada en JSON. Devuelve una cadena codificada en JSON en caso de éxito o FALSE en caso de error.

`json_last_error_msg()` — Devuelve el string con el error de la última llamada a `json_encode()` o a `json_decode()`

SINTAXIS

La sintaxis JSON está basada en los siguientes principios:

- los datos consisten en pares de nombre/valor
- los datos están separados por comas
- los objetos están definidos por llaves { }
- los arrays están definidos por corchetes []

Propiedades JSON:

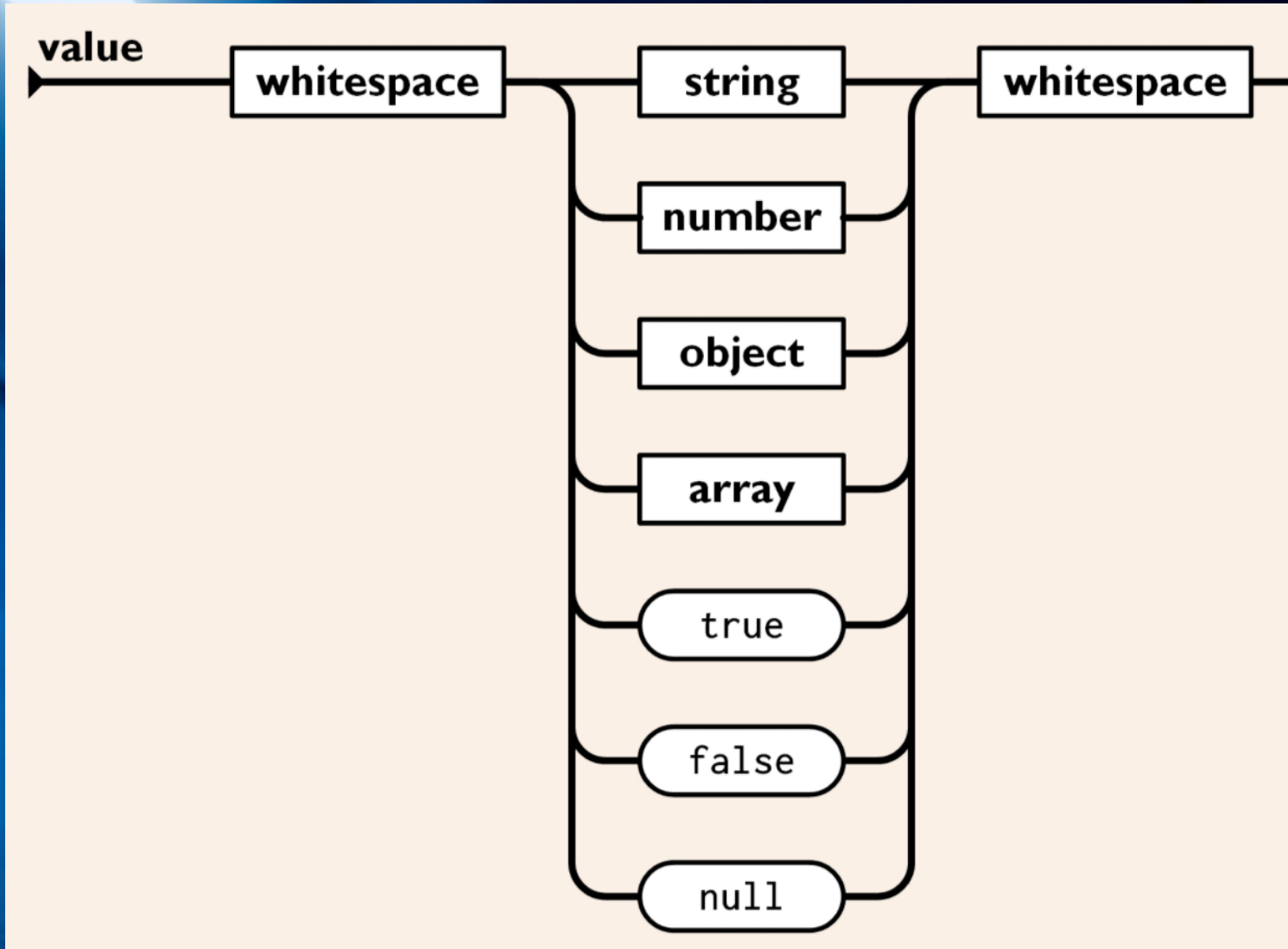
Los datos JSON se expresan en forma de pares (nombre/valor) (llave/valor) (clave/valor)

Un par nombre/valor contiene:

- un nombre de campo (entre comillas dobles) **NO se admiten comillas simples**
- luego dos puntos :
- seguido de un valor (entre comillas dobles) **NO se admiten comillas simples**

```
"Nombre" : "Juan "
```

Los nombres de propiedades(claves) **tienen en cuenta las mayúsculas y minúsculas**. Si escribes "Nombre" en lugar de "nombre", obtienes un nuevo par nombre/valor.



Tipos de
datos JSON

Tipos de datos JSON

Tipo	Descripción
cadena	Todo carácter excepto " y \
número	Entero o número de punto flotante
objeto	{ }
array	[]
booleano	true o false
nulo	null

true, false y null deben ir siempre en minúsculas, en caso contrario se generará un error

Un punto decimal(.) debe ir seguido de al menos un dígito

Una Cadena es una secuencia de cero o más caracteres Unicode.

Un Objeto es una colección desordenada de cero o más pares **nombre:valor**, donde un **nombre** es una cadena y un **valor** es una cadena, número, booleano, nulo, objeto o array.

Un Array es una secuencia ordenada de cero o más valores.

Tipos de datos JSON

El ejemplo que contiene todas las posibilidades de JSON

```
{
  "titulo": "Esto es un artículo",
  "visitas": 345,
  "publicado": true,
  "categoria": null,
  "precio" : 7.4,
  "comentarios": [
    {
      "autor": "Luisa López",
      "mensaje": "Muy buen artículo"
    },
    {
      "autor": "Carlos Pérez",
      "mensaje": "Artículo muy malo"
    }
  ]
}
```



Tipos de
datos JSON

json_encode()

TRADUCCIÓN DE PHP A JSON

Codificar JSON en PHP

```
json_encode(mixed $value, int $options = 0, int $depth = 512): string|false
```

Con `json_encode` se puede traducir cualquier cosa codificada en UTF-8 de PHP (salvo tipo `resources`) a un string JSON.

Todo se convierte en un objeto JSON, formando un par con clave y valor, excepto los arrays puros.

Un array puro en PHP, es aquel que contiene un índice numérico y ordenado, en este caso, se transforma en un array JSON

Las opciones `$options` permite cambiar la forma en que se codifican los datos.

Codificar JSON en PHP

```
json_encode(mixed $value, int $options = 0, int $depth = 512): string|false
```

\$options puede tomar los siguientes valores:

JSON_UNESCAPED_SLASHES : No escapar /

JSON_UNESCAPED_UNICODE : Codificar caracteres Unicode multibyte literalmente (por defecto es escapado como \uXXXX).

JSON_NUMERIC_CHECK: Codifica string numéricos como números.

JSON_FORCE_OBJECT: Devuelve un objeto en vez de un array cuando se usa un array no asociativo. Especialmente útil cuando el destinatario del resultado espera un objeto y el array está vacío.

Codificar JSON en PHP

Tipos básicos:

```
json_encode(array("Oso", "Perro", "Gato"));  
// Devuelve: ["Oso", "Perro", "Gato"];
```

```
json_encode(array(2=>"dos", 10=>"diez"));  
// Devuelve {"2":"dos", "10":"diez"}
```

```
json_encode(array("Oso"=> true, "Gato"=>null));  
// Devuelve: {"Oso":true, "Gato":null}
```

¿Cómo se traducen los arrays? Depende del índice que se utilice.

Puede verse que `json_encode()` traduce los tipos correctamente en el caso de booleano y null.

Codificar JSON en PHP

Incluye barras inclinadas / salientes en la salida

```
header("Content-Type:application/json;charset=utf-8");

$array = ["fichero" => "ejemplo.txt", "path" => "/proyecto/imagenes/fichero"];

//echo json_encode($array);

echo json_encode($array, JSON_UNESCAPED_SLASHES);
```

Salida: (Solo puedo enviar una salida)

```
//{"fichero":"ejemplo.txt","path":"/proyecto/imagenes/fichero"}

{"fichero":"ejemplo.txt","path":"/proyecto/imagenes/fichero"}
```


Codificar a JSON desde PHP

```
<?php
header('Content-Type:application/json;charset=utf-8');

//echo $a1 = json_encode(array("Oso", "Niña", "Ratón"));
//Devuelve:["Oso","Ni\u00f1a","Rat\u00f3n"];
//JSON codifica a Unicode, por defecto es escapado como \uXXXX

//No es recomendable en producción, sólo pruebas
//echo $a1 = json_encode(array("Oso", "Niña", "Ratón"), JSON_PRETTY_PRINT);

//Para forzarlo a trabajar en Unicode multibyte(UTF-8),
//Debemos añadir JSON_UNESCAPED_UNICODE
//echo $a2 = json_encode(array("Oso", "Niña", "Ratón"), JSON_UNESCAPED_UNICODE);

// Codifica string numéricos como números.
//echo $a3 = json_encode(array("Braveheart", "1995"), JSON_NUMERIC_CHECK | JSON_FORCE_OBJECT);
//Devuelve:{"0":"Braveheart","1":1995}

//No es un array puro porque NO está ordenado, obtenemos un objeto
//echo $a4 = json_encode(array(2 => "dos", 10 => "diez"));
//Devuelve{"2":"dos","10"=>"diez"}

//Array asociativo, también obtenemos un objeto
echo $a5 = json_encode(array("Oso" => true, "Gato" => null));
//Devuelve:{"Oso":true,"Gato":null}
```

Codificar JSON en PHP

```
<?php

$a1 = json_encode(array("Oso", "Niña", "Ratón"));
//Devuelve:["Oso","Ni\u00f1a","Rat\u00f3n"];
//JSON codifica a Unicode, por defecto es escapado como \uXXXX

//Para forzarlo a trabajar en Unicode multibyte(UTF-8),
//Debemos añadir JSON_UNESCAPED_UNICODE
$a2 = json_encode(array("Oso", "Niña", "Ratón"), JSON_UNESCAPED_UNICODE);

// Codifica string numéricos como números.
$a3 = json_encode(array("Braveheart", "1995"), JSON_NUMERIC_CHECK | JSON_FORCE_OBJECT);
//Devuelve:{"0":"Braveheart","1":1995}

//No es un array puro porque NO está ordenado, obtenemos un objeto
$a4 = json_encode(array(2 => "dos", 10 => "diez"));
//Devuelve{"2":"dos","10"=>"diez"}

//Array asociativo, también obtenemos un objeto
$a5 = json_encode(array("Oso" => true, "Gato" => null));
//Devuelve:{"Oso":true,"Gato":null}

//Si queremos crear un archivo json obtenido y guardarlo en el servidor:

file_put_contents("cosas1.json", $a1);
file_put_contents("cosas2.json", $a2);
file_put_contents("cosas3.json", $a3);
file_put_contents("cosas4.json", $a4);
file_put_contents("cosas5.json", $a5);
```

Codificar JSON en PHP

Casos poco frecuentes en los que no se crea el par clave-valor pero que son correctos en JSON

```
<?php
header('Content-Type:application/json;charset=utf-8');
//Observad que no hay par clave-valor en el resultado
$a = "una cadena";
$numero = 9.8;
echo $valor_string = json_encode($a);
//echo $valor_numerico = json_encode($numero);
?>
```

Objetos

```
<?php
header('Content-Type: application/json;charset=utf-8');

class Usuario
{
    // Nombre público, apellido vacío y fecha de nacimiento sin establecer
    public $nombre = "María";
    public $apellido = "";
    public $fechaNacimiento;
    public $colores=['rojo','verde','azul'];
    protected $direccion = "Calle Hispanidad";
    private $pais = "Mexico";
}
```


Objetos

```
$usuario = new Usuario();  
//$usuario->fechaNacimiento = date("Y-m-d H:i:s");  
$usuario_json= json_encode($usuario, JSON_UNESCAPED_UNICODE);  
echo $usuario_json;  
  
/*  
Devuelve:  
string '{"nombre":"María","apellido":"","fechaNacimiento":null,  
"colores":["rojo", "verde", "azul"]}'  
*/  
  
$file = 'usuarios.json';  
file_put_contents($file, $usuario_json);  
?>
```

json_decode()

TRADUCCIÓN DE JSON A PHP

Convierte un string codificado en JSON a una variable de PHP

```
json_decode(  
    string $json,  
    ?bool $associative = null,  
    int $depth = 512,  
    int $flags = 0  
): mixed
```

- Convierte un string codificado en JSON a una variable de PHP.
- Esta función sólo trabaja con string codificados en UTF-8.

\$associative :

- Cuando es **true**, los objects JSON devueltos serán convertidos a array asociativos
- Cuando es **false** los objects JSON devueltos serán convertidos a objetos.

Convierte un string codificado en JSON a una variable de PHP

La función `json_decode()` recibe una cadena codificada en JSON como su primer parámetro y la transforma en una variable de PHP.

`json_decode()` devolverá:

- Un objeto de tipo `stdClass` si el elemento de nivel superior de la cadena JSON es un objeto
- Una matriz indexada si el elemento superior JSON es una matriz.
- Los tipos de variables en JSON se convirtieron a su equivalente de PHP.
- Devuelve NULL cuando se produce un error.

Convierte un string codificado en JSON a una variable de PHP

La clase stdClass de PHP

En PHP existe una clase predefinida en el lenguaje que se llama stdClass. No tiene ni propiedades, ni métodos, ni padre; es una clase vacía.

¿Y para qué queremos esta clase si no contiene nada?

Podemos usar esta clase cuando necesitamos un objeto genérico al que luego le podremos añadir propiedades.

Esta clase genérica `stdClass` es la que utiliza la función `json_decode()` cuando tiene que traducir un string JSON a PHP que contiene un objeto.

```
<?php
```

```
$cadena_json = json_encode(array("Oso", "Niña", "Ratón"), JSON_UNESCAPED_UNICODE);
```

```
//Obtenemos un array numérico
```

```
var_dump($variable_php = json_decode($cadena_json));
```

```
$cadena_json = json_encode(array("Oso", "Niña", "Ratón"), JSON_UNESCAPED_UNICODE |  
JSON_FORCE_OBJECT);
```

```
//Obtenemos un objeto stdClass, observa que el array numérico se ha convertido en un objeto  
//con propiedades numéricas
```

```
var_dump($variable_php = json_decode($cadena_json));
```

```
$json_string = '{"nombre":"Pedro","edad":20,"activo":true,"colores":["rojo","azul"]}';
```

```
// Obtenemos un objeto stdClass
```

```
var_dump($variable_php = json_decode($json_string));
```

```
// El segundo parámetro debe ser true, si queremos que devuelva un array asociativo.
```

```
$array = json_decode($json_string, true);
```

```
var_dump($array);
```

```
?>
```


C:\xampp\htdocs\xClase\venancio\probarJSON\prueba7.php:4:

```
array (size=3)
  0 => string 'Oso' (length=3)
  1 => string 'Niña' (length=5)
  2 => string 'Ratón' (length=6)
```

C:\xampp\htdocs\xClase\venancio\probarJSON\prueba7.php:9:

```
object(stdClass)[1]
  public '0' => string 'Oso' (length=3)
  public '1' => string 'Niña' (length=5)
  public '2' => string 'Ratón' (length=6)
```

C:\xampp\htdocs\xClase\venancio\probarJSON\prueba7.php:14:

```
object(stdClass)[2]
  public 'nombre' => string 'Pedro' (length=5)
  public 'edad' => int 20
  public 'activo' => boolean true
  public 'colores' =>
    array (size=2)
      0 => string 'rojo' (length=4)
      1 => string 'azul' (length=4)
```

C:\xampp\htdocs\xClase\venancio\probarJSON\prueba7.php:18:

```
array (size=4)
  'nombre' => string 'Pedro' (length=5)
  'edad' => int 20
  'activo' => boolean true
  'colores' =>
    array (size=2)
      0 => string 'rojo' (length=4)
      1 => string 'azul' (length=4)
```

Convierte un string codificado en JSON a una variable de PHP

Para conseguir una matriz asociativa para objetos JSON en lugar de obtener un objeto, hay que utilizar `true` como segundo parámetro a `json_decode()` .

```
$json_string = '{"nombre": "Pedro", "edad": 20, "activo": true, "colores": ["rojo", "azul"]}';  
  
// El segundo parámetro debe ser true  
$array = json_decode($json_string, true);  
  
var_dump($array);
```

Forzamos a que convierta los datos en array asociativo, aunque el string JSON sea un objeto.

Convierte un string codificado en JSON a una variable de PHP

```
<?php  
  
$json_string = '{"nombre": "Pedro", "edad": 20, "activo": true, "colores": ["rojo", "azul"]}';  
  
var_dump($variable_php = json_decode($json_string, true));  
  
?>
```

C:\wamp64\www\clase\json\cuarto.php:18:

array (*size=4*)

'nombre' => string **'Pedro'** (*length=5*)

'edad' => int **20**

'activo' => boolean **true**

'colores' =>

array (*size=2*)

0 => string **'rojo'** (*length=4*)

1 => string **'azul'** (*length=4*)